S2D-99: Best Practice Summary

> **Series:** S2D | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 04/03/2026

Overview

This notebook consolidates every actionable best practice from the S2D (Splunk to Dynatrace) migration series into a single definitive reference. Each practice specifies the exact setting, value, or action to apply.

Table of Contents

1. Migration Planning
2. Log Discovery and Validation
3. Query Translation (SPL to DQL)
4. Alert Migration - Davis Anomaly Detectors
5. Alert Migration - Workflows
6. Extended Timeframes (ArrayMovingSum)
7. Metric Creation from Logs
8. Dashboard Migration
9. Naming Standards
10. DQL Performance and Syntax

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`logs.read`, `settings.write`, `documents.write`, `automation.write`
Splunk Access	Ability to view existing queries, alerts, dashboards
Knowledge	Familiarity with both Splunk SPL and Dynatrace DQL

1. Migration Planning

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Follow the 5-phase migration sequence	Phase 1: Discovery, Phase 2: Data Validation, Phase 3: Query Translation, Phase 4: Alert Migration, Phase 5: Dashboard Migration	Critical	Process
2	Inventory all Splunk assets before starting	Document every dashboard, alert, report, and saved search with their SPL queries	Critical	Process
3	Prioritize by criticality	Migrate production monitoring first, then staging, then dev	Recommended	Process
4	Map Splunk indexes to Dynatrace buckets	Each Splunk index maps to a Grail bucket; document the mapping before translation	Critical	Process
5	Validate log availability before query translation	Run DQL validation queries (S2D-02) before attempting SPL-to-DQL conversion	Critical	Process
6	Compare log volumes between platforms	Run equivalent time-bounded queries in both Splunk and Dynatrace to confirm counts match	Recommended	Validation

2. Log Discovery and Validation

#	Best Practice	Recommended Setting/Value	Priority	Category
7	Set log ingest rules at the host-group level	Use host-group scope, not per-host	Recommended	Configuration
8	Enable OneAgent log content access	`oneagentctl --set-log-content-access=true` on every monitored host	Critical	Configuration
9	Configure custom log sources for non-standard paths	Settings > Log Monitoring > Custom log sources; specify the exact file path pattern	Recommended	Configuration
10	Configure timestamp and splitting rules for multi-line logs	Settings > Log Monitoring > Timestamp and splitting; set the timestamp pattern and line separator	Recommended	Configuration
11	Validate by host name first, then by log source path	Use `matchesPhrase(host.name, "...")` then `matchesPhrase(log.source, "...")`	Recommended	Validation

#	Best Practice	Recommended Setting/Value	Priority	Category
12	For Kubernetes, validate by cluster then namespace then deployment	Filter chain: `k8s.cluster.name` -> `k8s.namespace.name` -> `k8s.deployment.name`	Recommended	Validation
13	Remember: host-level OneAgent settings override environment settings	If log monitoring is disabled on the OneAgent, environment-level ingest rules will not apply	Critical	Configuration

3. Query Translation (SPL to DQL)

#	Best Practice	Recommended Setting/Value	Priority	Category
14	Use `==` for equality, not `=`	`filter field == "value"`	Critical	Syntax
15	Use double quotes only, not single quotes	`"value"` not `'value'`	Critical	Syntax
16	Use curly-brace arrays, not parentheses	`in(field, {"a", "b"})` not `field IN ('a', 'b')`	Critical	Syntax
17	Translate `search index=...` to `fetch logs`	`fetch logs, from:-1h` with explicit time range	Critical	Syntax
18	Translate `where` to `filter`	`filter field == "value"`	Critical	Syntax
19	Translate `stats` to `summarize`	`summarize count = count(), by:{field}`	Critical	Syntax
20	Translate `eval` to `fieldsAdd`	`fieldsAdd new_field = expression`	Critical	Syntax
21	Translate `table` to `fields`	`fields field1, field2`	Recommended	Syntax
22	Translate `sort -field` to `sort field desc`	Explicit `asc` / `desc` keyword	Recommended	Syntax
23	Translate `head N` to `limit N`	`limit 100`	Recommended	Syntax
24	Translate `rename old AS new` to `fieldsRename`	`fieldsRename old, alias:new`	Recommended	Syntax
25	Translate `timechart span=5m count by level` to `makeTimeseries`	`makeTimeseries count = count(), by:{loglevel}, interval:5m`	Critical	Syntax
26	Translate `rex` field extraction to `parse` with DPL	`parse content, "LD 'user=' WORD:username"`	Recommended	Syntax

#	Best Practice	Recommended Setting/Value	Priority	Category
27	Translate `*value*` wildcard to `matchesPhrase()`	`matchesPhrase(field, "value")` for token-based matching	Critical	Syntax
28	Translate `field IN ("a","b")` to `in()` function	`in(field, {"a", "b"})`	Critical	Syntax
29	Translate `NOT field="value"` to `!=` or `filterOut`	`filter field != "value"` or `filterOut field == "value"`	Recommended	Syntax
30	Always alias aggregation results	`summarize count = count()` not `summarize count()`	Critical	Syntax

4. Alert Migration - Davis Anomaly Detectors

#	Best Practice	Recommended Setting/Value	Priority	Category
31	Apply the threshold translation formula	`Splunk Threshold = DT Threshold x DT Violating Samples`	Critical	Alerting
32	Use the "Alert Reimagined" strategy as default	Threshold: calculated, Sliding Window: match Splunk timeframe, Violating Samples: 2-3, Dealerting Samples: match Splunk suppress duration	Recommended	Alerting
33	Set Davis Anomaly Detector query interval to 1 minute	`interval:1m` in `makeTimeseries`	Critical	Alerting
34	Set the sliding window to match Splunk query timeframe	Max: 60 minutes. If Splunk timeframe is 15m, set sliding window to 15	Critical	Alerting
35	Set dealerting samples to match Splunk suppression	If Splunk suppress = 15 min, set dealerting samples = 15	Recommended	Alerting
36	Use `by: {dt.entity.cloud_application}` for entity association	Ensures alerts are tied to the correct monitored entity	Critical	Alerting
37	Prefer Davis Anomaly Detectors over Workflows when timeframe is 60 min or less	Davis provides continuous monitoring, AI correlation, and no license overhead	Critical	Alerting

#	Best Practice	Recommended Setting/Value	Priority	Category
38	If alert timeframe exceeds 60 minutes, use Workflows or ArrayMovingSum	Davis Anomaly Detectors max sliding window = 60 minutes	Critical	Alerting
39	Use the data-driven strategy when historical data is available	Analyze 7 days of data to set thresholds based on actual patterns	Recommended	Alerting
40	Validate the alert query returns timeseries data before configuring	Run the query in a notebook first; confirm `makeTimeseries` output	Critical	Validation

5. Alert Migration - Workflows

#	Best Practice	Recommended Setting/Value	Priority	Category
41	Use Workflows only when: timeframe > 60 min, alert runs few times/day, business-hours only, or is actually a report	Otherwise use Davis Anomaly Detectors	Critical	Alerting
42	Specify timeframe explicitly in the DQL query, not from dashboard	`fetch logs, from:now()-7d` in the workflow query itself	Critical	Alerting
43	Workflow queries do NOT require `makeTimeseries` output	Use `summarize` to return a single aggregated value	Recommended	Alerting
44	Set event timeout in the JavaScript event creation step	`timeout: 15` (minutes) to auto-expire events	Recommended	Alerting
45	Workflow events do NOT auto-close or auto-update	Design event lifecycle accordingly; events expire by timeout only	Critical	Alerting
46	Workflow events are NOT correlated by Davis AI	No root-cause analysis or problem grouping	Recommended	Alerting
47	Account for workflow license consumption	Workflows consume workflow hours; frequent schedules cost more	Recommended	Licensing

#	Best Practice	Recommended Setting/Value	Priority	Category
48	For business-hours-only alerting, filter by hour in the Davis query instead of using a Workflow	`fieldsAdd hour = toLong(formatTimestamp(timestamp, format:"HH"))` then `filter hour >= 8 AND hour < 18`	Recommended	Alerting

6. Extended Timeframes (ArrayMovingSum)

#	Best Practice	Recommended Setting/Value	Priority	Category
49	Use `arrayMovingSum(array, 60)` with `interval:1m` for 60-minute rolling sum	Each data point = sum of preceding 60 minutes	Critical	DQL
50	Max `arrayMovingSum` window = 60	Cannot exceed 60 data points regardless of interval	Critical	DQL
51	With Davis Anomaly Detectors + ArrayMovingSum, set sliding window = 1 and violating samples = 1	Each data point already contains the full rolling aggregation	Critical	Alerting
52	Set threshold to total count expected in the rolling window	If Splunk threshold = 500 errors in 60 min, set DT threshold = 500	Critical	Alerting
53	Remove the original timeseries field after adding the rolling sum	`fieldsRemove error_count` after `fieldsAdd error_count_1h = arrayMovingSum(error_count, 60)`	Recommended	DQL
54	For dashboard visualizations > 60 min, increase the interval	`interval:4m` with `window:60` = 4-hour rolling sum	Recommended	DQL
55	For alerts > 60 minutes that cannot use proportional reduction, use Workflows	If error distribution is uneven, proportional threshold reduction will produce false positives	Recommended	Alerting

7. Metric Creation from Logs

#	Best Practice	Recommended Setting/Value	Priority	Category
56	Create log-based metrics for frequently executed dashboard and alert queries	Reduces query cost, improves speed, increases retention	Recommended	Performance
57	Metric filter must use only supported functions	`matchesPhrase`, `matchesValue`, `isNull`, `isNotNull`, basic operators only. No `parse` or `fieldsAdd`	Critical	Configuration
58	Avoid high-cardinality dimensions	Never use transaction IDs, session IDs, request IDs, or timestamps as metric dimensions	Critical	Configuration
59	Consolidate similar metrics using dimensions instead of separate metrics	One `service_errors` metric with `k8s.deployment.name` dimension, not per-service metrics	Recommended	Design
60	Name metrics as `log.[app_name].[metric_description]`	Example: `log.easytravel.error_count`	Recommended	Naming
61	Complete all SPL-to-DQL translation before requesting metric extraction	Metric filters must match the final DQL query filters	Critical	Process
62	Query extracted metrics with `timeseries` not `fetch logs`	`timeseries avg(log.easytravel.error_count), from:-1h`	Critical	Syntax

8. Dashboard Migration

#	Best Practice	Recommended Setting/Value	Priority	Category
63	Layout: top row = single-value KPIs, middle = time-series trends, bottom = detail tables	Standard 3-tier dashboard layout	Recommended	Design
64	Use dashboard variables for interactive filtering	Define `$cluster`, `$namespace`, `$deployment` variables with query-backed dropdowns	Recommended	Design

#	Best Practice	Recommended Setting/Value	Priority	Category
65	Populate dropdown variables with DQL	`fetch logs, from:-1h | summarize count(), by:{field} | fields field | sort field asc`	Recommended	Design
66	Use consistent time intervals across related charts on the same dashboard	All trend charts should use the same `interval:` value	Recommended	Design
67	Replace Splunk gauges with single-value tiles + color thresholds	Dynatrace has no gauge visualization; use conditional formatting	Recommended	Design
68	Create Log Searcher dashboards for investigation	VM version: variables for `host_name` and `log_source`. K8s version: variables for `cluster`, `namespace`, `deployment`	Recommended	Design
69	Validate migrated dashboard data against Splunk	Compare identical time ranges side-by-side before decommissioning Splunk dashboards	Critical	Validation

9. Naming Standards

#	Best Practice	Recommended Setting/Value	Priority	Category
70	Dashboard names	`[app_name] Dashboard Title`	Critical	Naming
71	Alert configuration names	`[app_name] alert name from splunk`	Critical	Naming
72	Alert event names (triggered title)	`[app_name] [priority] – {dimension} – alert name` e.g. `[EasyTravel] P2 – {host.name} – High Error Count`	Critical	Naming
73	Report names	`[Report] [app_name] Report Title`	Recommended	Naming
74	Lookup table paths	`/lookups/[app_name]/[table_name]`	Recommended	Naming
75	Log-based metric names	`log.[app_name].[metric_description]` (lowercase, underscores)	Recommended	Naming
76	Alerting workflow names	`[Alert] [app_name] Alert Description`	Recommended	Naming

#	Best Practice	Recommended Setting/Value	Priority	Category
77	Automation workflow names	`[Auto] [app_name] Automation Description`	Recommended	Naming
78	Remediation workflow names	`[Remediate] [app_name] Remediation Description`	Optional	Naming
79	Enforce naming during migration, not after	Review every asset name at creation time	Critical	Process
80	Use infrastructure-as-code to enforce naming	Monaco or Terraform templates with naming prefixes built in	Recommended	Process

10. DQL Performance and Syntax

#	Best Practice	Recommended Setting/Value	Priority	Category
81	Always specify a time range on `fetch`	`fetch logs, from:-1h` (never bare `fetch logs`)	Critical	Performance
82	Filter immediately after `fetch`	Place `filter` commands before any `fieldsAdd`, `parse`, or `summarize`	Critical	Performance
83	Use `==` for exact matches, `matchesPhrase` for token search	`==` is faster than `~` or `matchesPhrase` when the full value is known	Recommended	Performance
84	Target specific Grail buckets	`fetch logs, bucket: {"app_logs_*"}` to avoid scanning all buckets	Recommended	Performance
85	Use named parameters for DQL functions	`round(value, decimals: 2)` not `round(value, 2)`	Critical	Syntax
86	Use `countIf()` instead of nested filter + count	`summarize errors = countIf(loglevel == "ERROR")` in a single pass	Recommended	Performance
87	Check `isNotNull()` before aggregating optional fields	`filter isNotNull(db.system)` before `summarize avg(duration), by:{db.system}`	Recommended	Correctness
88	Sort and limit must come last in the pipeline	`sort field desc | limit 10` as final commands	Critical	Performance

#	Best Practice	Recommended Setting/Value	Priority	Category
89	Use `fieldsKeep` or `fieldsRemove` early to drop unneeded columns	Reduces data volume through the pipeline	Recommended	Performance
90	Use `samplingRatio:` for exploratory queries on large datasets	`fetch logs, samplingRatio:100` then multiply results back	Optional	Performance

Summary

This notebook contains **90 best practices** across 10 categories for Splunk to Dynatrace migration:

Category	Count	Priority Breakdown
Migration Planning	6	4 Critical, 2 Recommended
Log Discovery and Validation	7	3 Critical, 4 Recommended
Query Translation (SPL to DQL)	17	10 Critical, 7 Recommended
Alert Migration - Davis Anomaly Detectors	10	7 Critical, 3 Recommended
Alert Migration - Workflows	8	3 Critical, 5 Recommended
Extended Timeframes (ArrayMovingSum)	7	4 Critical, 3 Recommended
Metric Creation from Logs	7	3 Critical, 4 Recommended
Dashboard Migration	7	1 Critical, 6 Recommended
Naming Standards	11	4 Critical, 6 Recommended, 1 Optional
DQL Performance and Syntax	10	4 Critical, 5 Recommended, 1 Optional
Total	**90**	**43 Critical, 45 Recommended, 2 Optional**

References

- **S2D-01** - Getting Started: Migration overview and planning
- **S2D-02** - Locating Logs: Data validation and ingest configuration
- **S2D-03** - SPL to DQL: Query translation fundamentals
- **S2D-04** - Davis Anomaly Detectors: Continuous alert migration
- **S2D-05** - Workflow Alerts: Scheduled and extended alerting
- **S2D-06** - ArrayMovingSum: Extended timeframe handling
- **S2D-07** - Metric Creation: OpenPipeline log-based metrics
- **S2D-08** - Dashboard Migration: Visualization conversion

- **S2D-09** - Naming Standards: Asset organization conventions

---